



分类与预测-回归

2026/4/23

分类与预测——回归分析

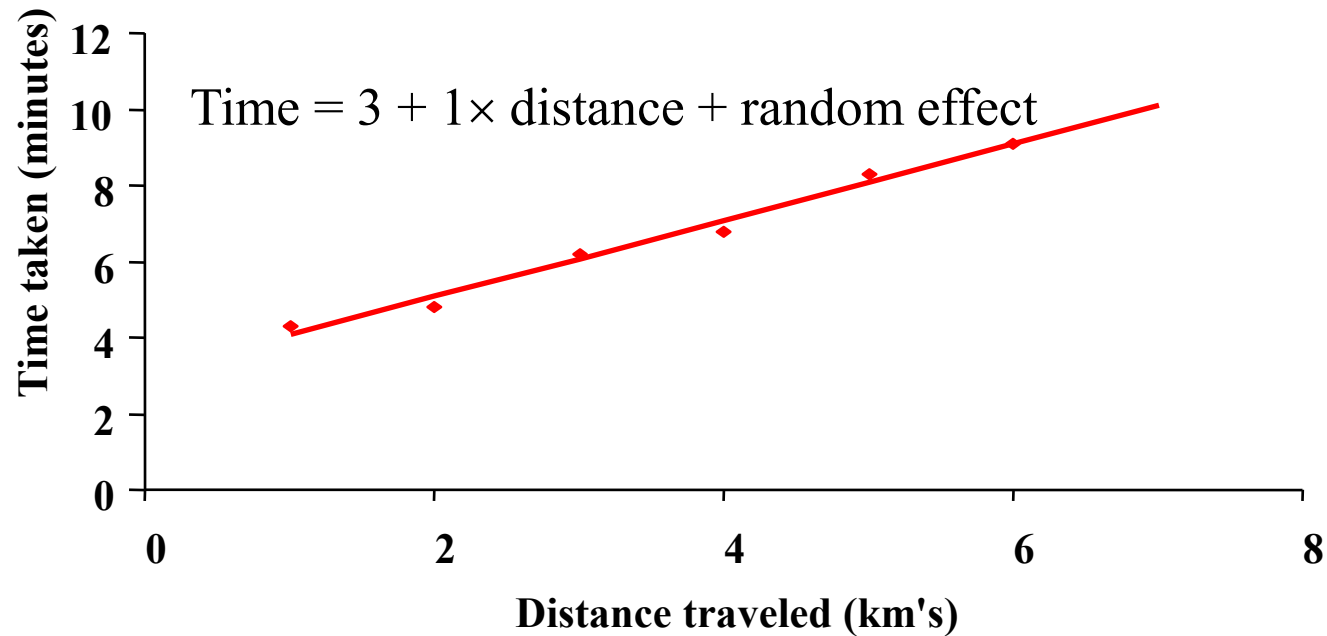
- 回归分析是通过建立模型来研究变量之间相互关系的密切程度、结构状态及进行模型预测的一种有效工具，在工商管理、经济、社会、医学和生物学等领域应用十分广泛。



- 从19世纪初高斯提出最小二乘估计算法起，回归分析的历史已有200多年

回归——Linear Regression线性回归分析

$$Y = \theta_0 + \theta_1 X_1 + \theta_2 X_2 + \cdots + \theta_n X_n + \varepsilon$$



基于旅行距离来预测旅行时间

回归——Linear Regression线性回归分析

- y 为预测值，连续变量 $X_1=(x_{1,1}, x_{1,2}, \dots, x_{1,k})$ ，之间的关系可以建模为

$$y = \theta_0 + \theta_1 x_{1,1} + \theta_2 x_{1,2} + \dots + \theta_k x_{1,k} + \varepsilon$$

- 给定数据 x 及 y , 我们可以估计 $\theta_0, \theta_1, \theta_2, \dots, \theta_k$

$$y = X\theta$$
$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$
$$X = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} & \cdot & \cdot & \cdot & x_{1,k} \\ 1 & x_{2,1} & x_{2,2} & \cdot & \cdot & \cdot & x_{2,k} \\ \cdot & \cdot & \cdot & & & & \cdot \\ \cdot & \cdot & \cdot & & & & \cdot \\ \cdot & \cdot & \cdot & & & & \cdot \\ 1 & x_{i,1} & x_{i,2} & \cdot & \cdot & \cdot & x_{i,k} \\ \cdot & \cdot & \cdot & & & & \cdot \\ \cdot & \cdot & \cdot & & & & \cdot \\ \cdot & \cdot & \cdot & & & & \cdot \\ 1 & x_{n,1} & x_{n,2} & \cdot & \cdot & \cdot & x_{n,k} \end{bmatrix}$$
$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \cdot \\ \cdot \\ \cdot \\ \theta_k \end{bmatrix}$$

- 损失函数—衡量估计的误差

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (x_i \theta - y_i)^2 = \frac{1}{2n} \|X\theta - y\|_2^2 = \frac{1}{2n} (X\theta - y)^T (X\theta - y)$$

回归——Linear Regression线性回归分析

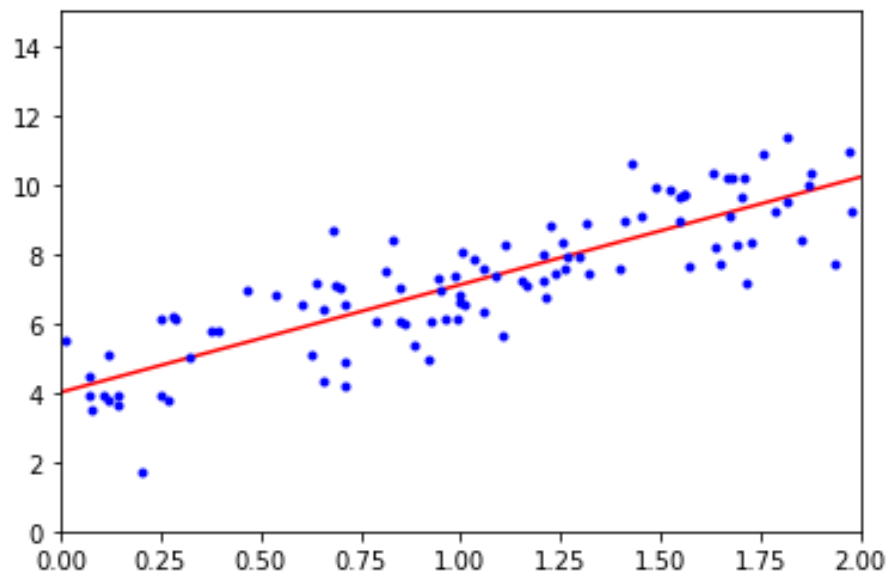
解析解

由于 $J(\theta)$ 的二阶导 $X^T X$ 为半正定矩阵，所以Hessian矩阵是半正定的

对 $J(\theta)$ 求偏导并令其为0， $X^T X \hat{\theta} - X^T y = 0$ ，得到的解就是使损失函数最小的解，我们得到

$$\hat{\theta} = (X^T X)^{-1} X^T y$$

```
import numpy as np
import matplotlib.pyplot as plt
X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)
#生成模拟数据
X_b = np.c_[np.ones((100, 1)), X]
beta_best = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)
print(beta_best)
X_new = np.array([[0], [2]])
X_new_b = np.c_[np.ones((2, 1)), X_new]
print(X_new_b)
y_predict = X_new_b.dot(beta_best)
print(y_predict)
# 绘制函数图像和测试集点分布
plt.plot(X_new, y_predict, 'r-')
plt.plot(X, y, 'b.')
plt.axis([0, 2, 0, 15])
plt.show()
```



不足：

- $X^T X$ 是 $d \times d$ 维的，求逆矩阵的计算复杂度是 d 的三次方
- 如果 $X^T X$ 是奇异阵（行列式为零），则无法得到解

回归——Linear Regression线性回归分析

梯度下降 (GD)

```
import numpy as np
import matplotlib.pyplot as plt
X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)
X_b = np.c_[np.ones((100, 1)), X]
n_epochs = 500
t0, t1 = 5, 50
m = 100
```

```
def learning_schedule(t):
```

```
    return t0 / (t + t1)
```

```
beta = np.random.randn(2, 1)
```

```
for epoch in range(n_epochs):
```

```
    for i in range(m):
```

```
        random_index = np.random.randint(m)
```

```
        xi = X_b[random_index:random_index+1]
```

```
        yi = y[random_index:random_index+1]
```

```
        gradients = 2 * xi.T.dot(xi.dot(beta) - yi)
```

```
        learning_rate = learning_schedule(epoch * m + i)
```

```
        beta = beta - learning_rate * gradients
```

```
X_new = np.array([[0], [2]])
```

```
X_new_b = np.c_[np.ones((2, 1)), X_new]
```

```
print(X_new_b)
```

```
y_predict = X_new_b.dot(beta)
```

```
print(y_predict)
```

```
# 绘制函数图像和测试集点分布
```

```
plt.plot(X_new, y_predict, 'r-')
```

```
plt.plot(X, y, 'b.')
```

```
plt.axis([0, 2, 0, 15])
```

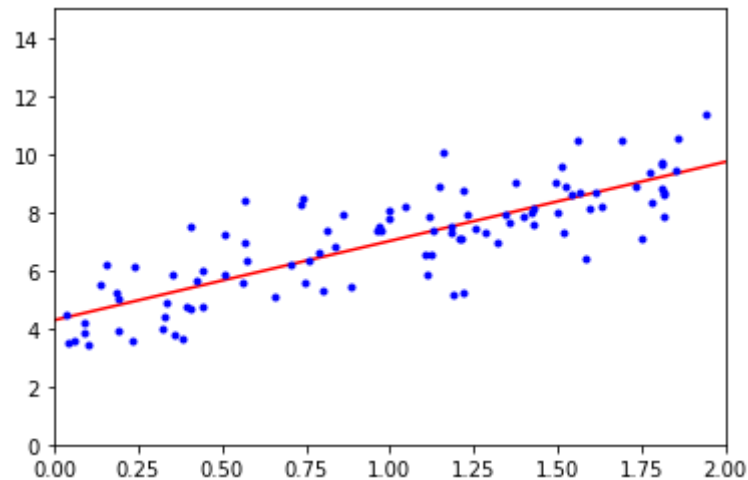
```
plt.show()
```

当距离最优解很远时，我们希望学习率大些，加快学习速度，当距离最优解较近是，我们希望减小学习率，避免来回震荡

$$\theta = \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$$
$$\frac{\partial J(\theta)}{\partial \theta} = \frac{\partial}{\partial \theta} \frac{1}{2n} \sum_{i=1}^n (x_i \theta - y_i)^2 = \frac{1}{n} \sum_{i=1}^n (x_i \theta - y_i) x_i$$
$$\theta_j = \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j} = \theta_j - \alpha \frac{1}{n} \sum_{i=1}^n (x_i \theta - y_i) x_i$$



学习率



学习到的模型

回归——偏差 (Bias) 和方差 (Variance)

在监督学习中，已知样本 $(x_1, y_1), \dots, (x_n, y_n)$ ，要求拟合出一个模型（函数 $\hat{f}$ ），其预测值 $\hat{f}(x)$ 与样本实际值 y 的误差最小。

考虑到样本数据其实是采样， y 并不是真实值本身，假设真实模型（函数）是 f ，则采样值 $y = f(x) + \varepsilon$ ，其中 ε 代表噪音，其均值为 0，方差为 σ^2 。

拟合函数 $\hat{f}$ 的主要目的是希望它能对新的样本进行预测，所以，拟合出函数 $\hat{f}$ 后，需要在测试集（训练时未见过的数据）上检测其预测值与实际值 y 之间的误差。可以采用平方误差函数（mean squared error）来度量其拟合的好坏程度，即 $(y - \hat{f}(x))^2$

$$E[(y - \hat{f}(x))^2] \quad \text{误差的期望值} = \text{噪音的方差} + \text{模型预测值的方差} + \text{预测值相对真实值的偏差的平方}$$

$$= \sigma^2 \quad \text{噪声方差: 标记值与真实值差平方的期望}$$

$$+ \text{Var}[\hat{f}(x)] \quad \text{方差: 使用样本数相同的不同训练集产生的方差}$$

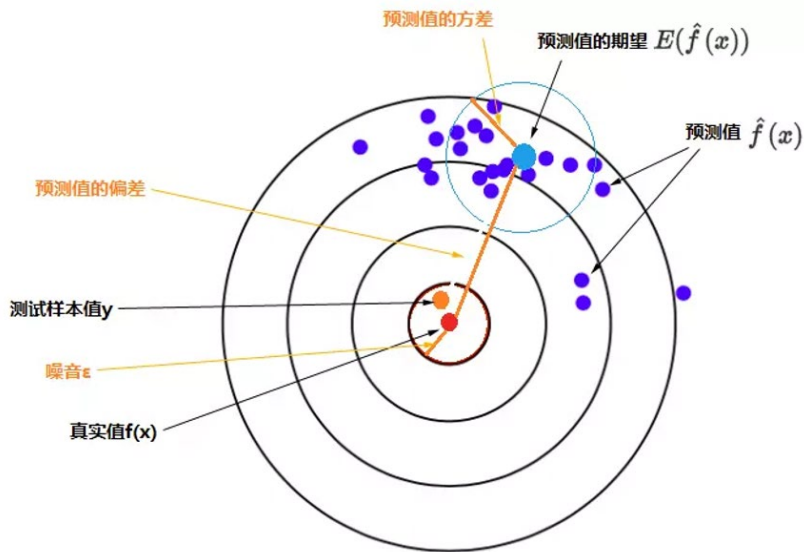
$$+ (\text{Bias}[\hat{f}(x)])^2 \quad \text{偏差: 期望输出与真实标记的差别}$$

$$\text{Bias}[\hat{f}(x)] = f(x) - E[\hat{f}(x)]$$

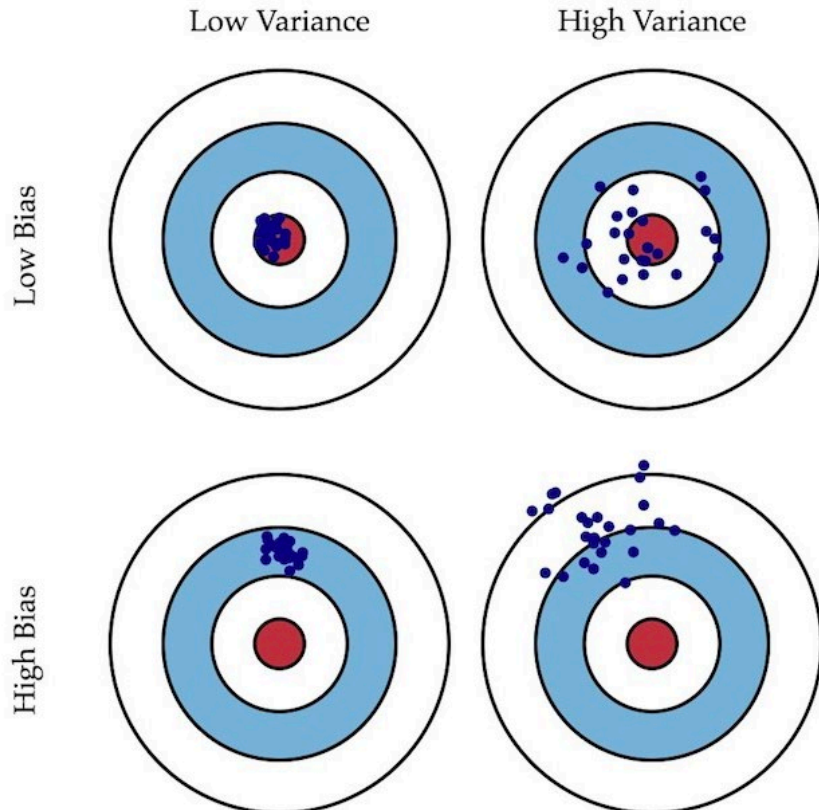
- 噪声则表示任何学习算法在泛化能力的下界，描述了学习问题本身的难度；
- 方差度量了模型预测值的变化，描述了数据扰动造成的影响；
- 偏差度量了学习算法的期望预测与真实结果的偏离程度，刻画描述了算法本身对数据的拟合能力，也就是训练数据的样本与训练出来的模型的匹配程度。

$$\begin{aligned} E[(y - \hat{f})^2] &= E[y^2 + \hat{f}^2 - 2y\hat{f}] \\ &= E[y^2] + E[\hat{f}^2] - E[2y\hat{f}] \\ &= (\text{Var}[y] + (E[y])^2) + (\text{Var}[\hat{f}] + (E[\hat{f}])^2) - E[2(f + \varepsilon)\hat{f}] \\ &= \text{Var}[y] + \text{Var}[\hat{f}] + (E[y])^2 + (E[\hat{f}])^2 - E[2f\hat{f} + 2\varepsilon\hat{f}] \\ &= \text{Var}[y] + \text{Var}[\hat{f}] + f^2 + (E[\hat{f}])^2 - E[2f\hat{f}] - E[2\varepsilon\hat{f}] \\ &= \text{Var}[y] + \text{Var}[\hat{f}] + f^2 + (E[\hat{f}])^2 - 2fE[\hat{f}] - 2E[\varepsilon]E[\hat{f}] \\ &= \text{Var}[y] + \text{Var}[\hat{f}] + (f^2 + (E[\hat{f}])^2 - 2fE[\hat{f}]) \\ &= \text{Var}[y] + \text{Var}[\hat{f}] + (f - E[\hat{f}])^2 \\ &= \sigma^2 + \text{Var}[\hat{f}] + (\text{Bias}[\hat{f}])^2 \end{aligned}$$

回归——偏差 (Bias) 和方差 (Variance)



$$\begin{aligned} E[(y - \hat{f}(x))^2] \\ = \sigma^2 \\ + \text{Var}[\hat{f}(x)] \\ + (\text{Bias}[\hat{f}(x)])^2 \end{aligned}$$



选择相对较好的模型的顺序:

方差小, 偏差小

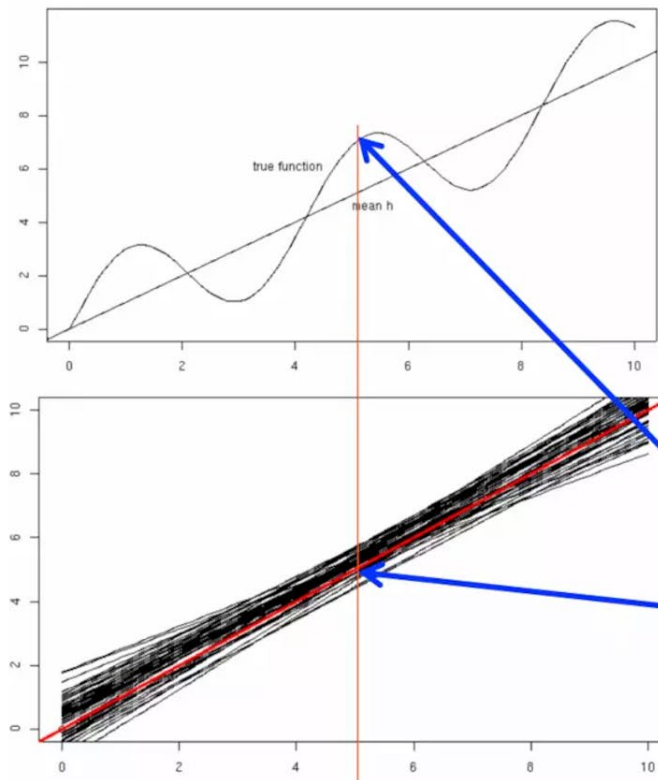
方差小, 偏差大 ; 方差大, 偏差小

方差大, 偏差大。

回归——偏差和方差的计算

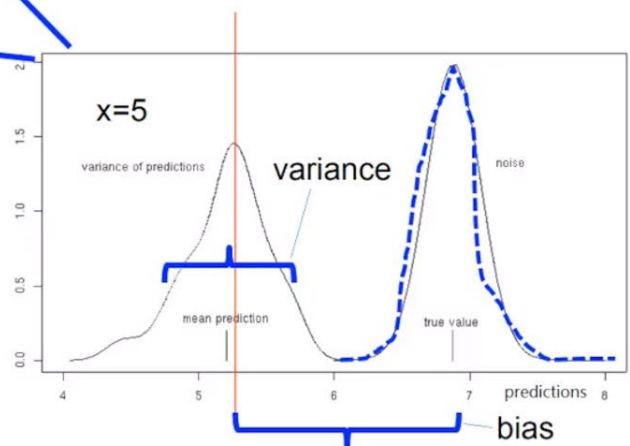
$$y = x + 2 \sin(1.5x) + N(0,0.2)$$

真实模型及生成的样本



用线性函数来构建模型，每次取20个样本来训练一个线性模型，总共构造50个线性模型

取 $x=5$ 来计算偏差及方差，如下图所示



回归——影响因素

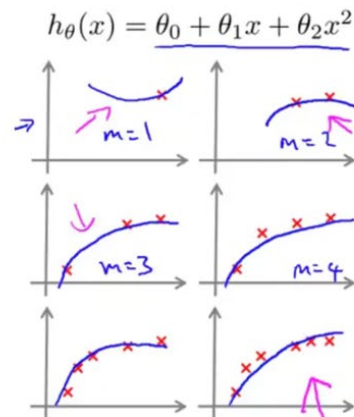
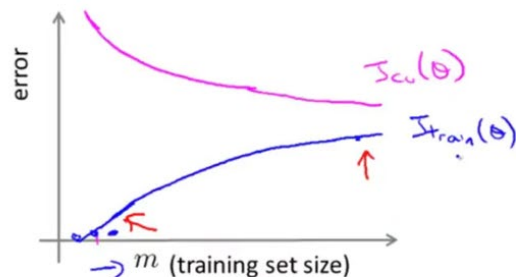
- 样本数量
- 多项式回归
- 正则化项

回归——样本的数量

Learning curves

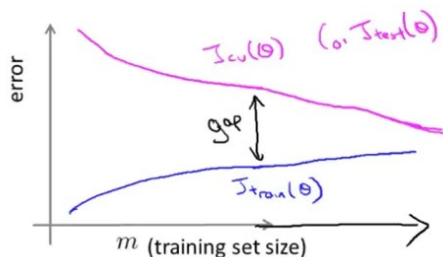
$$J_{train}(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$J_{cv}(\theta) = \frac{1}{2m_{cv}} \sum_{i=1}^{m_{cv}} (h_{\theta}(x_{cv}^{(i)}) - y_{cv}^{(i)})^2$$



Andrew Ng

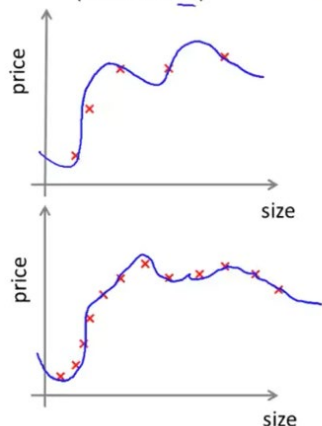
High variance



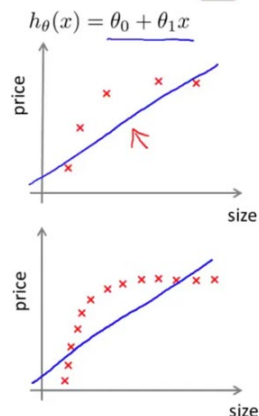
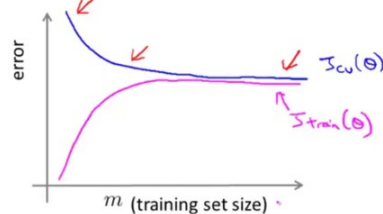
If a learning algorithm is suffering from high variance, getting more training data is likely to help. $\leftarrow$

$$h_{\theta}(x) = \theta_0 + \theta_1 x + \dots + \theta_{100} x^{100}$$

(and small λ)



High bias



Andrew Ng

模型方差较高时，增加样本会有帮助

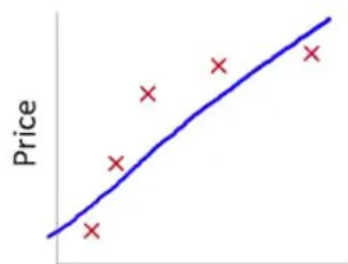
模型偏差较高时，增加样本帮助不大

分类与预测——回归分析

回归模型	算法描述	
线性回归	对一个或多个自变量和因变量之间的线性关系进行建模，可用最小二乘法求解模型系数。	
非线性回归	对一个或多个自变量和因变量之间的非线性关系进行建模。如果非线性关系可以通过简单的函数变换转化成线性关系，用线性回归的思想求解；如果不能转化，用非线性最小二乘方法求解。	
岭回归	是一种改进最小二乘估计的方法。	$L(\theta) = J(\theta) + \lambda \ \theta\ _2^2$
套索回归	Lasso estimate具有shrinkage (收缩方法又称为正则化 regularization) 和selection (选择部分重要特征) 两种功能	$L(\theta) = J(\theta) + \eta \ \theta\ _1$
ElasticNet回归	ElasticNet是Lasso和Ridge回归技术的混合体。它使用L1来训练并且L2优先作为正则化矩阵。	$L(\theta) = J(\theta) + \lambda \ \theta\ _2^2 + \eta \ \theta\ _1$

回归——Polynomial Regression多项式回归

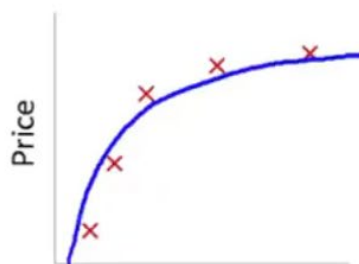
Bias/variance



$$\theta_0 + \theta_1 x$$

High bias
(underfit)

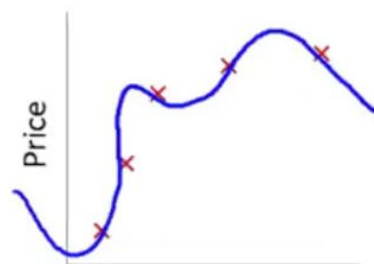
$$d=1$$



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

“Just right”

$$d=2$$



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

High variance
(overfit)

$$d=4$$

多项式次数	模型复杂度	方差	偏差	过/欠拟合
低	低	低	高	欠拟合
中	中	中	中	适度
高	高	高	低	过拟合

回归——Ridge Regression岭回归

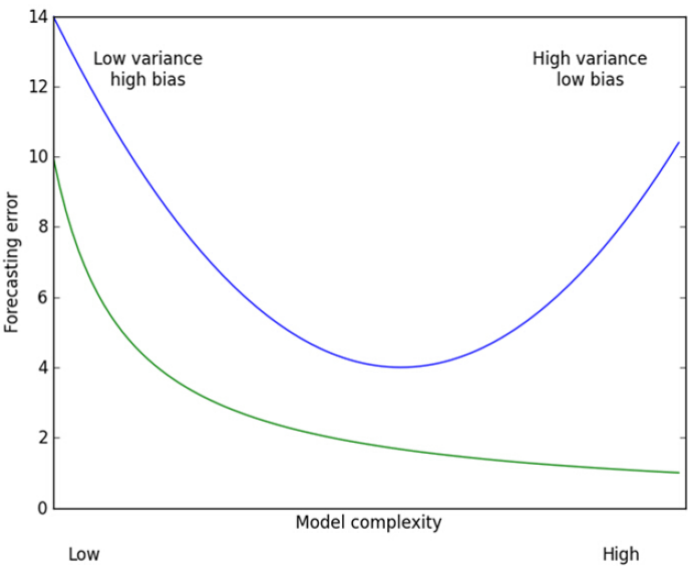
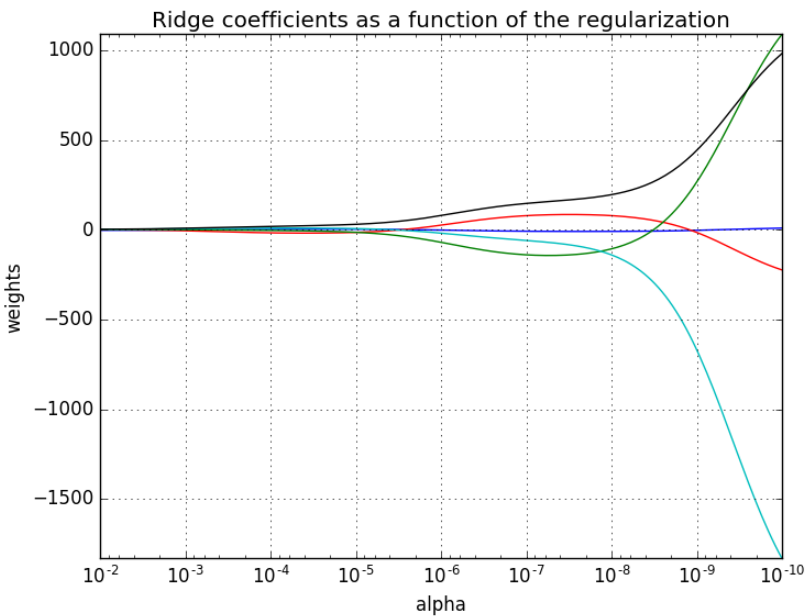


Figure 8.8 The bias variance tradeoff illustrated with test error and training error. The training error is the top curve, which has a minimum in the middle of the plot. In order to create the best forecasts, we should adjust our model complexity where the test error is at a minimum.

<http://blog.csdn.net/google19890102>

$$L(\theta) = J(\theta) + \lambda \|\theta\|_2^2 = J(\theta) + \lambda \sum_{j=1}^k \theta_j^2$$

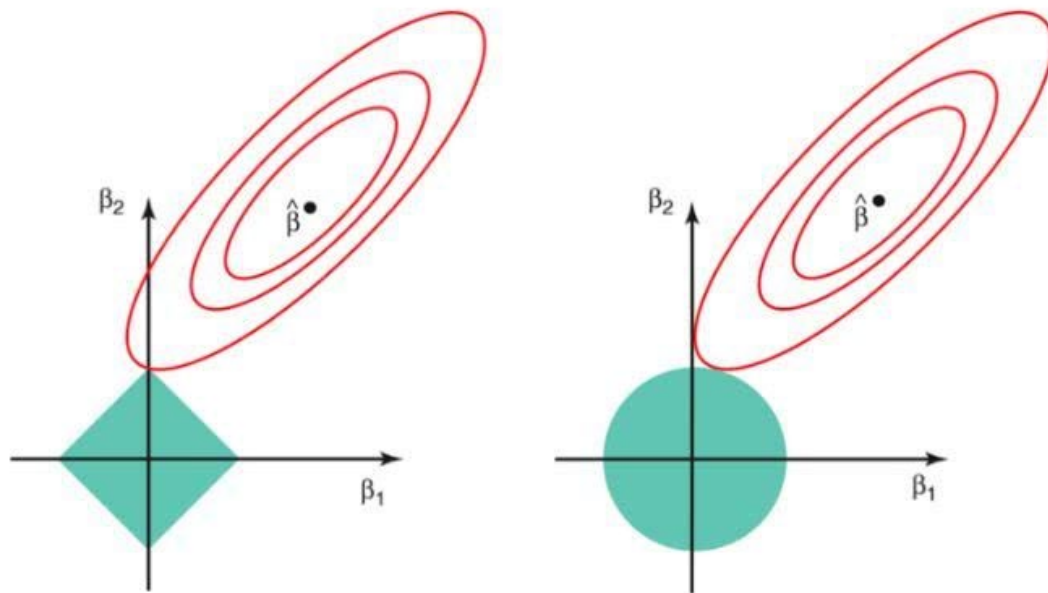


正则化项权重λ	模型复杂度	方差	偏差	过/欠拟合
大	低	低	高	欠拟合
中	中	中	中	适度
小	高	高	低	过拟合

回归——Lasso Regression套索回归

Lasso estimate具有shrinkage(收缩方法又称为正则化regularization)和selection(选择部分重要特征)两种功能

$$L(\theta) = J(\theta) + \beta \|\theta\|_1 = J(\theta) + \beta \sum_{j=1}^k |\theta_j|$$



Credit : An Introduction to Statistical Learning by Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani

重要优点:

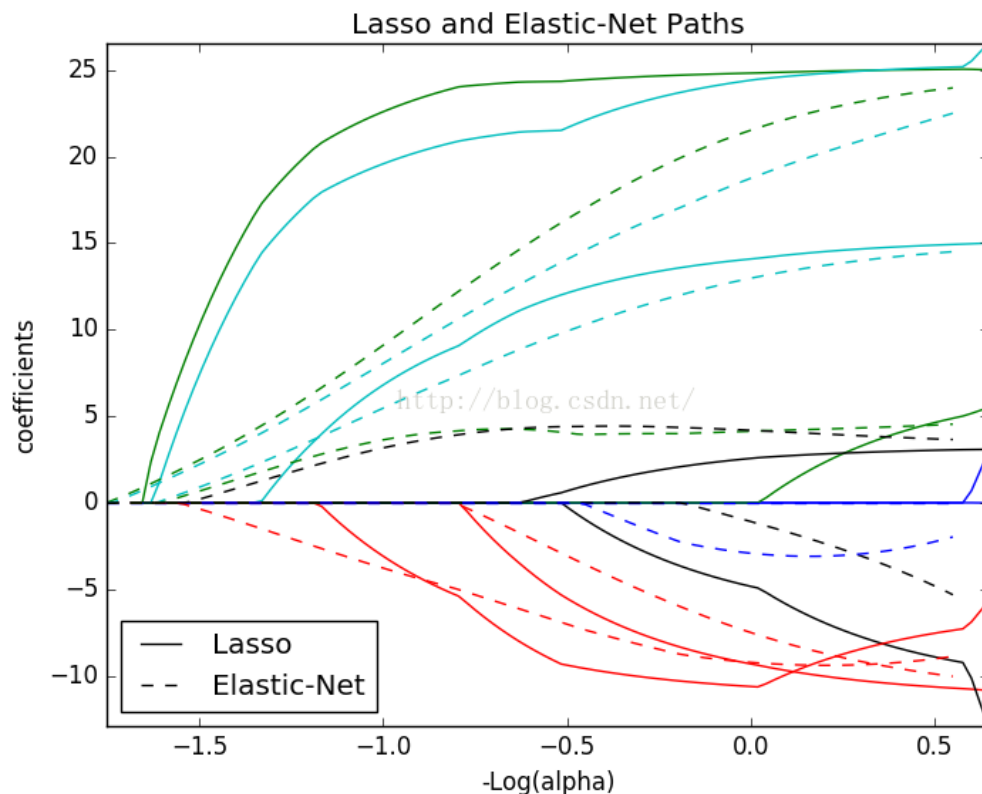
- (1) 这种回归在模型假设方面与岭回归完全一致;
- (2) 模型可将某些变量对应的系数直接收缩至0, 这有助于变量的筛选;
- (3) 该模型属于L1惩罚;
- (4) 如果存在某组预测因子高度相关的情况, Lasso方法仅会选取它们中的一个, 并把所对应的系数收缩至0。

回归——ElasticNet回归

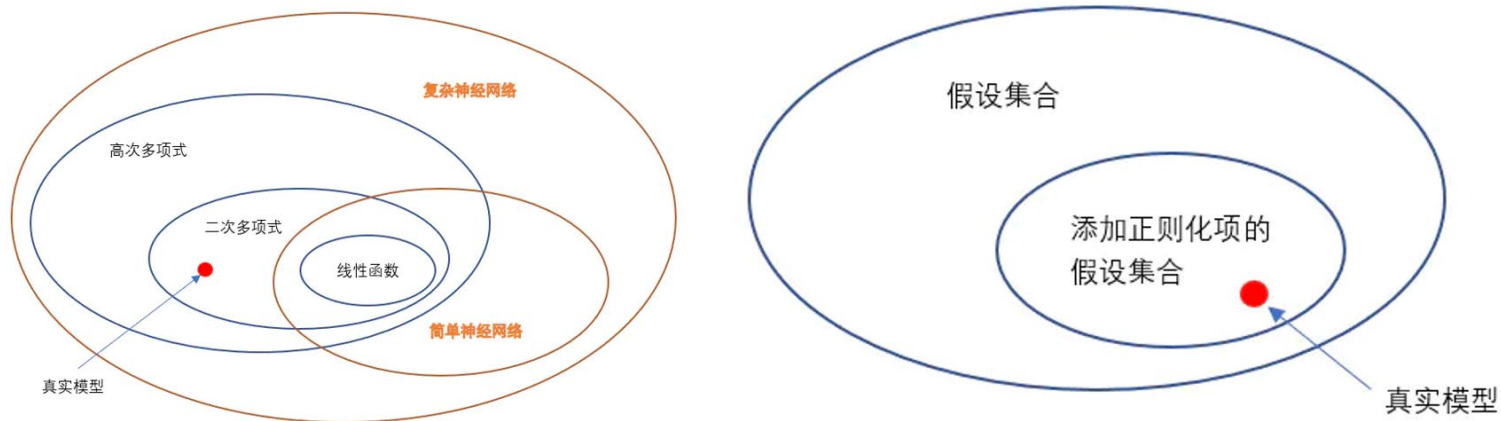
ElasticNet是Lasso和Ridge回归技术的混合体。它使用L1来训练并且L2优先作为正则化矩阵。

$$L(\theta) = J(\theta) + \alpha \|\theta\|_2^2 + \beta \|\theta\|_1$$

ElasticNet在我们发现用Lasso回归太过(太多特征被稀疏为0), 而岭回归也正则化的不够(回归系数衰减太慢)的时候, 可以考虑使用ElasticNet回归来达到一个折衷的效果。



回归——选择假设集合



有监督学习的整个流程，其实就是一个不断缩小假设集合的过程。从大的方面看可以分为两个步骤：

- 选择一个假设集合，包括模型及相关结构、超参数等。选择一些不同的假设集合，然后通过考察它们的偏差方差，对各假设集合的性能进行评估。
- 使用样本数据进行训练，使该模型尽量拟合样本，就是从上面选定的假设集合中找到一个特定的假设（模型）。

1. 线性回归实例

1) 导入相关工具包

```
import numpy as np
from sklearn import linear_model
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split # 用于分割训练集和测试集
from sklearn.preprocessing import PolynomialFeatures # 多项式特征
import matplotlib.pyplot as plt
import pandas as pd
```

2) 读取数据

```
# 读取数据
x = np.random.uniform(-3, 3, size=100)
# print(x.shape)
y = 0.5 * x ** 2 + x + 2 + np.random.normal(0, 1, size=100)
# plt.scatter(x, y)
# plt.show()
x, y = x.reshape(-1, 1), y.reshape(-1, 1)

# 数据切片分割
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.8) # 分为训练集和测试集数据，test样本占比80%
```

3) 添加多项式特征

```
# 使用多项式回归
ploy = PolynomialFeatures(degree=2) # 添加一个最高次项为2的多项式特征
x_train_new = ploy.fit_transform(x_train)
# 将原本数据集 X 的每一种特征，转化为对应的多项式的特征，生成的多项式特征相应的数据集赋值给新的数据集
x_test_new = ploy.fit_transform(x_test)
```

4) 建立线性模型并拟合

```
# 使用一元线性回归
lin_reg = linear_model.LinearRegression()
lin_reg.fit(x_train, y_train)
```

5) 得到预测结果

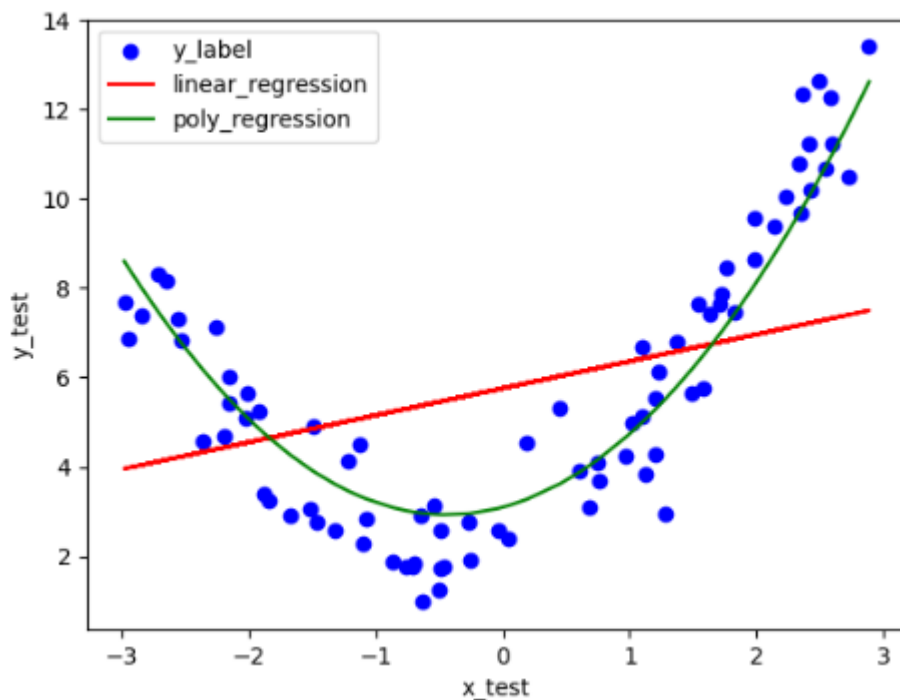
```
y_predict = lin_reg.predict(x_test)
print("mse:", mean_squared_error(y_predict, y_test))

# 预测y值
y_predict = liner_model.predict(x_test_new)
# 模型评价：求模型的预测值与真实值均方误差，接近0的效果好
print("mse2:", mean_squared_error(y_predict, y_test)) # MSE: 均方根误差
```

6) 可视化

```
# 可视化
plt.scatter(x_test, y_test, c='blue') #对测试集数据描点
plt.plot(x_test, y_predict, c="red")
```

```
# plt.scatter(x_test, y_test) # 对测试集数据描点
plt.plot(np.sort(x_test.reshape(-1)), y_predict.reshape(-1)[np.argsort(x_test.reshape(-1))], c="green")
plt.legend(['y_label', 'linear_regression', 'poly_regression'])
plt.xlabel('x_test')
plt.ylabel('y_test')
plt.show()
```



MSE (均方根误差):

线性回归: 8.923905809097793

多项式回归: 1.134025267229244

2. Ridge、Lasso、ElasticNet回归实例

1) 导入相关工具包

```
from sklearn.linear_model import Ridge
import numpy as np
from matplotlib import pyplot as plt
import sklearn.datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn import linear_model
```

2) 读取数据

```
x, y = sklearn.datasets.make_regression(n_features=1, noise=5, random_state=2020)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.8)
```

3) 建立多种线性模型

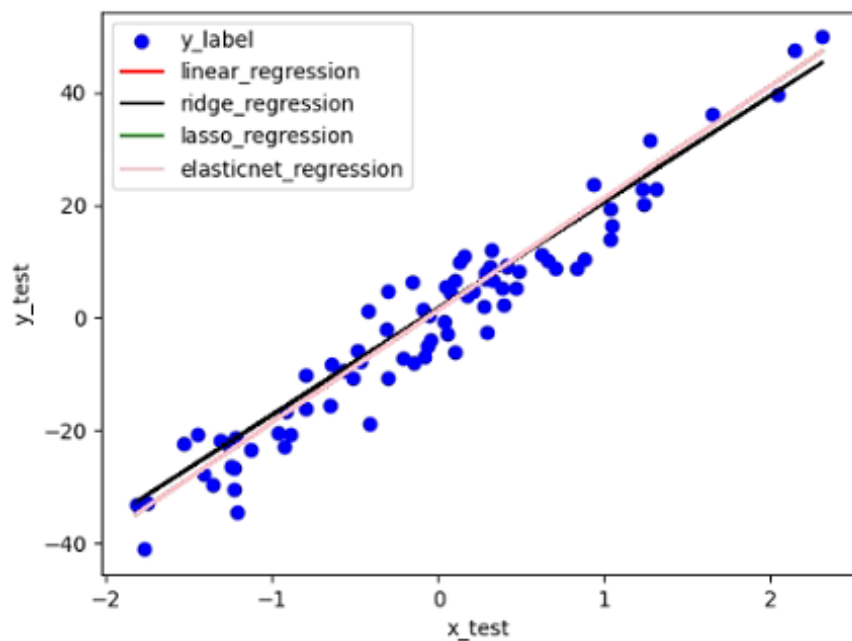
```
lin_reg = linear_model.LinearRegression()
lin_reg.fit(x_train, y_train)
ridge_reg = linear_model.Ridge(alpha=1)
ridge_reg.fit(x_train, y_train)
las_reg = linear_model.Lasso(alpha=10)
las_reg.fit(x_train, y_train)
ela_reg = linear_model.ElasticNet(alpha=0.01)
ela_reg.fit(x_train, y_train)
```

4) 拟合并得到预测结果

```
y_predict_linear = lin_reg.predict(x_test)
print("mse linear:", mean_squared_error(y_predict_linear, y_test))
y_predict_ridge = ridge_reg.predict(x_test)
print("mse ridge:", mean_squared_error(y_predict_ridge, y_test))
y_predict_las = lin_reg.predict(x_test)
print("mse lasso:", mean_squared_error(y_predict_las, y_test))
y_predict_ela = lin_reg.predict(x_test)
print("mse elasticnet:", mean_squared_error(y_predict_ela, y_test))
```

5) 可视化

```
plt.scatter(x_test, y_test, c='blue') # 对测试集数据描点
plt.plot(x_test, y_predict_linear, c='red')
plt.plot(x_test, y_predict_ridge, c='black')
plt.plot(x_test, y_predict_lasso, c='green')
plt.plot(x_test, y_predict_elasticnet, c='pink')
plt.legend(['y_label', 'linear_regression', 'ridge_regression', 'lasso_regression', 'elasticnet_regression'])
plt.xlabel('x_test')
plt.ylabel('y_test')
plt.show()
```



MSE (均方根误差):

线性回归: 23.898973173839504

Ridge 回归 (alpha=1): 25.761683506281503

Lasso 回归 (alpha=10): 23.898973173839504

Elasticnet 回归 (alpha=0.01): 23.898973173839504

回归分类——Logistic回归

- Logistic回归属于概率型非线性回归，分为二分类和多分类的回归模型。对于二分类的Logistic回归，因变量 y 只有“是、否”两个取值，记为1和0。假设在自变量作用下， y 取“是”的概率是 p ，则取“否”的概率是 $1-p$ ，研究的是当 y 取“是”发生的概率 p 与自变量的关系。

邮件：垃圾邮件Spam / 非垃圾邮件Not Spam?

在线交易：欺诈 (Yes / No)?

癌症Tumor: 恶性Malignant / 良性Benign?

一般性假设： 0: 负类“Negative Class” 1: 正类“Positive Class”

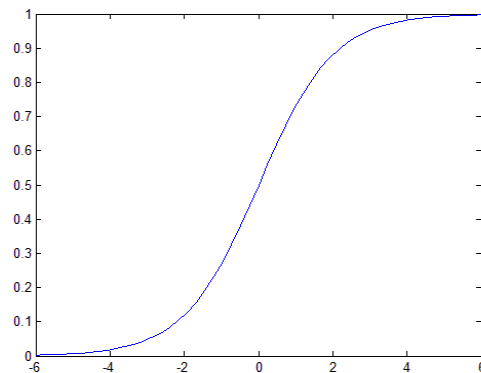
- Logistic函数。

二分类的Logistic回归模型中的因变量的只有1-0（如是和否、发生和不发生）两种取值。假设 m 个独立自变量 $x_1, x_2, \dots, x_m$ 作用下，记 y 取1的概率是 $p = P(y = 1|X)$ ，取0概率是 $1-p$ ，取1和取0的概率之比为 $\frac{p}{1-p}$ ，称为事件的优势比（odds）

，对odds取自然对数即得Logistic变换： $Logit(p) = \ln\left(\frac{p}{1-p}\right)$

当 p 在 $(0, 1)$ 之间变化时，odds的取值范围是 $(0, +\infty)$ ， $Logit(p)$ 的取值范围是

$(-\infty, +\infty)$ ，令 $Logit(p) = \ln\left(\frac{p}{1-p}\right) = z$ ，则 $p = \frac{1}{1+e^{-z}}$ ，即为Logistic函数，如下图：



回归分类——Logistic回归

- Logistic回归模型

Logistic回归模型是建立 $\ln\left(\frac{p}{1-p}\right)$ 与自变量的线性回归模型。

Logistic回归模型为： $\ln\left(\frac{p}{1-p}\right) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_k x_k + \varepsilon$

因为 $\ln\left(\frac{p}{1-p}\right)$ 的取值范围是 $(-\infty, +\infty)$ ，这样，自变量 $\theta_1, \theta_2, \dots, \theta_k$ 可在任意范围内取值。

记 $h_\theta(x) = \frac{1}{1+e^{\theta_0+\theta_1 x_1+\theta_2 x_2+\dots+\theta_k x_k+\varepsilon}}$ ，得到：

$$p = P(y = 1|X) = h_\theta(x) \quad 1 - p = P(y = 0|X) = 1 - h_\theta(x)$$

Cost function: $J(\theta) = \frac{1}{n} \sum_{i=1}^n \text{Cost}(h_\theta(x_i), y_i) = -\frac{1}{n} \sum_{i=1}^n (y_i \log h_\theta(x_i) + (1 - y_i) \log(1 - h_\theta(x_i)))$

用类似线性回归的梯度下降方法进行优化

为什么逻辑回归损失函数不用MSE
<https://www.jianshu.com/p/af1e5cff21b9>

回归分类——逻辑回归分类示例

1. 导入相关库

```
from sklearn.linear_model import  
LogisticRegression  
from sklearn.model_selection import  
train_test_split  
from sklearn.metrics import accuracy_score,  
confusion_matrix
```

2. 读取并处理数据

```
iris = datasets.load_iris()  
x = iris.data[:, :2]  
y = iris.target  
X_train, X_test, y_train, y_test =  
train_test_split(x, y, random_state=666)
```

3. 训练模型并得到预测结果

```
# 用pipeline建立模型  
# StandardScaler()作用：去均值和方差归一化。  
# 且是针对每一个特征维度来做的，而不是针对样本。  
# StandardScaler对每列分别标准化。  
# LogisticRegression () 建立逻辑回归模型  
lr = Pipeline([('sc', StandardScaler()), ('clf',  
LogisticRegression())])  
lr.fit(X_train, y_train)
```

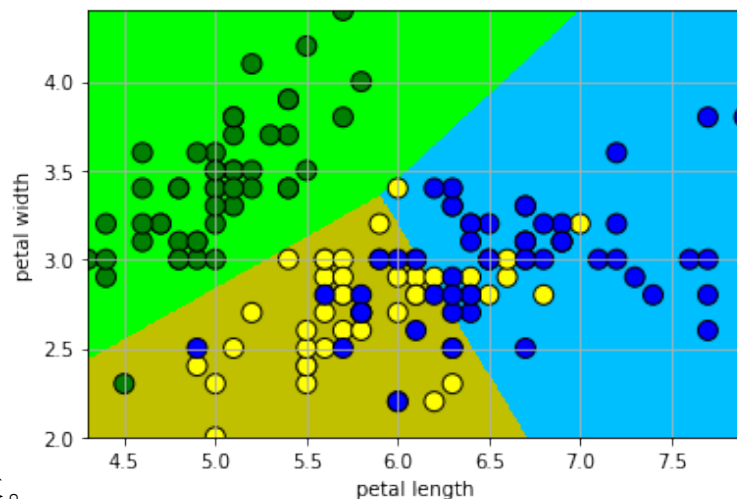
```
# 使用测试集预测
```

```
y_pred = model.predict(X_test)
```

```
# 计算准确性
```

```
accuracy = accuracy_score(y_test, y_pred)
```

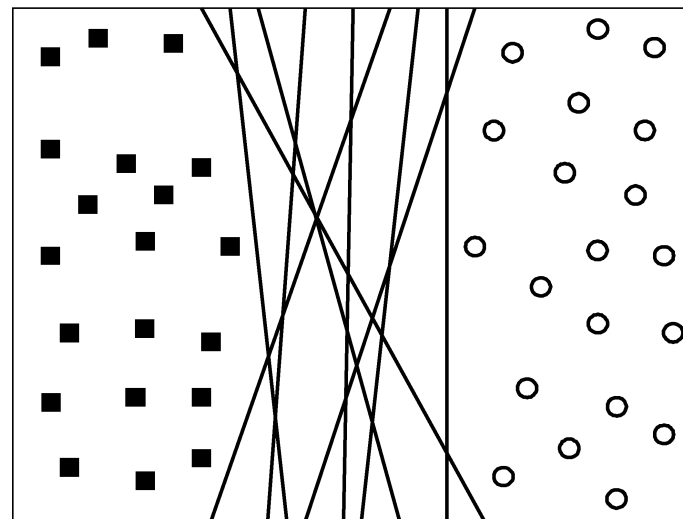
```
print("准确性: ", accuracy)
```



回归分类——SVM

◆ 两个线性可分的类

- 找到这样一个超平面，使得所有的方块位于这个超平面的一侧，而所有的圆圈位于它的另一侧
- 可能存在无穷多个那样的超平面

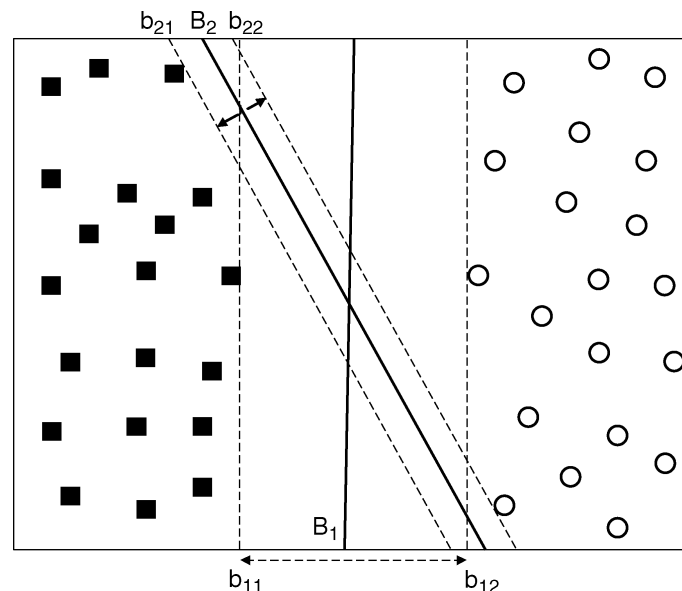


◆ 找这样的超平面，它最大化边缘

- B1比B2好

$$\min \frac{1}{2} \|\mathbf{w}\|^2$$

$$\text{约束条件} \quad y_i(\mathbf{w}\mathbf{x}_i + b) \geq 1 \quad i=1,2,\dots,N$$



回归分类——SVM

- 使用Karuch-Kuhn-Tucher (KKT) 条件:

$$\lambda_i \geq 0$$

$$\lambda_i [y_i(\mathbf{w}\mathbf{x}_i + b) - 1] = 0$$

(5.42)

- 除非训练实例满足方程 $y_i(\mathbf{w}\mathbf{x}_i + b) = 1$, 否则拉格朗日乘子 λ_i 必须为零
- $\lambda_i > 0$ 的训练实例位于超平面 b_{i1} 或 b_{i2} 上, 称为支持向量
- (5.39) 和 (5.40) 代入到公式 (5.38) 中
$$L_D = \sum_{i=1}^N \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j \quad (5-43)$$
$$\text{s.t. } \sum_{i=1}^N \lambda_i y_i = 0$$
- 这是 L_p 的对偶问题(最大化问题)。可以使用数值计算技术, 如二次规划来求解 λ

回归分类——SVM

- 解出 λ_i 后, 用(5.39)求 $\mathbf{w}$, 再用(5.42)求 b
- 决策边界为

$$\left(\sum_{i=1}^N \lambda_i y_i \mathbf{x}_i \cdot \mathbf{x} \right) + b = 0$$

- $\mathbf{z}$ 可以按以下的公式来分类

$$f(\mathbf{z}) = \text{sign}(\mathbf{w}\mathbf{z} + b) = \text{sign}\left(\sum_{i=1}^N \lambda_i y_i \mathbf{x}_i \cdot \mathbf{z} + b\right)$$

- 如果 $f(\mathbf{z}) = 1$, 则检验实例 $\mathbf{z}$ 被分为到正类, 否则分到负类

回归分类——SVM

- 如果不是线性可分的，怎么办？

- 软边缘（soft margin）

- 允许一定训练误差

- 引入松弛变量 ξ_i

$$\min \frac{1}{2} \|\mathbf{w}\|^2 + C \left(\sum_{i=1}^N \xi_i \right)^k$$

约束条件 $y_i(\mathbf{w}\mathbf{x}_i + b) \geq 1 - \xi_i, \quad i = 1, 2, \dots, N$

- 其中 C 和 k 是用户指定的参数，
对误分训练实例加罚

- 取 $k=1$,

C 根据模型在确认集上的性能选择

